



C++ Programming

(335)

State 2025

PRODUCTION:

State_CPlusPlus

_____ (530 points)

Test Time: 90 minutes

RUBRIC

Solution and Project (There is NO partial credit) (NOTE: UC represents uppercase and LC represents lowercase)		
The project file is present on the flash drive in a single folder with your contest ID		10 points
Program Execution (If the program does not execute, then the remaining items in this section receive a score of zero)		
When launched the program starts with no errors		20 points
The beginning of the print out has the “BPA Speeding Ticket Database” form header; the format matches the sample provided in the packet with less than two deviations		10 points
There are 12 total records that have printed out with the record header the increments by 1 for each print out.		10 points
All 12 record headers are identical accept for the incrementing number and the format matches the example provided with less than two deviations		10 points
Every record includes the filled descriptors exactly as shown in the sample output.		10 points
The data and the records is printed in the exact the order in which it is listed in the “Names.txt” file		50 points
The format for all the records matches the sample output exactly. This includes all spacing and the order of information.		50 points
The form footer is included after the 12th record is printed it matches the sample output exactly.		10 points
Subtotal		/180 Points

Source Code Review (There is NO partial credit) No credit will be given for missing flagged comments Code segments must work to receive full credit		
A comment containing the contestant number is present at the top of the CPP_State.cpp file.		10 points
SC1: Flag Comment Present		5 points
SC1: class Name the constructor handles parameters for the first and last name		15 points
SC2: Flag Comment Present		5 points
SC2: class Name method getFullName() returns the first and last name together.		10 points
SC3: Flag Comment Present		5 points
SC3: class Name has no public variables		10 points
SC4: Flag Comment Present		5 points
SC4: class CarInfo the constructor handles parameters for the car make, car model, and license plate.		15 points
SC5: Flag Comment Present		5 points
SC5: class CarInfo method getInfo() returns the car make, car model and license plate together.		10 points
SC6: Flag Comment Present		5 points
SC6: class CarInfo class has no public variables		10 points
SC7: Flag Comment Present		5 points
SC7: class Payment the constructor handles parameters for the ticket payment amount and credit card number.		15 points
SC8: Flag Comment Present		5 points
SC8: class Payment method returns the formatted payment value for USD using stringstream and setprecision and it also returns the credit card number.		15 points
SC9: Flag Comment Present		5 points
SC9: class Payment class uses no public variables		10 points
SC10: Flag Comment Present		5 points
SC10: class Customer the constructor handles parameters for the following objects: Name, CarInfo, and Payment.		10 points
SC11: Flag Comment Present		5 points
SC11: class Customer method creates the field descriptors for the three objects and gets the proper information from each object by using the object getter methods.		10 points
SC12: Flag Comment Present		5 points
SC12: class Customer class has no public variables.		10 points
SC13: Flag Comment Present		5 points

SC13: class Customer class method run() controls the program by initiating the reading of the files and printing the records using readDataFromFile() and printRecords().		10 points
SC14: Flag Comment Present		5 points
SC14: TicketDriver class method readDataFromFile() finds Names.txt and gives error message if the file cannot open properly		15 points
SC15: Flag Comment Present		5 points
SC15: TicketDriver class method readDataFromFile() uses stringstream that reads each line and looks for commas and assigns values to all of the variables using a loop structure.		40 points
SC16: Flag Comment Present		5 points
SC16: SC15: TicketDriver class method readDataFromFile() uses error handling for parsing error, AND properly closes file once completed.		20 points
SC17-15: Flag Comment Present		5 points
SC17: SC15: TicketDriver class method readDataFromFile() properly closes file once completed.		20 points
SC18: Flag Comment Present		5 points
SC18: TicketDriver class method void printTickets(): method uses a loop to print all available tickets.		20 points
SC19: Flag Comment Present		5 points
SC19: TicketDriver class method void printTickets(): method uses a loop to update the counter on the record header (remember it will count from 1 to 12).		10 points
Subtotal		/380 Points
Total Points		/ Points

```
..._State\Grader Files\CPP_State\CPP_State\CPP_State.cpp 1
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <sstream>
5 #include <iomanip>
6
7 using namespace std;
8
9 // Define the classes
10
11 class Name {
12 public: //SC1
13     Name(const string& first, const string& last) : firstName(first),
14         lastName(last) {}
15     string getFullName() const {
16         return firstName + " " + lastName; //SC2
17     }
18 private: //SC3
19     string firstName;
20     string lastName;
21 };
22
23 class CarInfo {
24 public: //SC4
25     CarInfo(const string& make, const string& model, const string& plate)
26         : carMake(make), carModel(model), licensePlate(plate) {}
27     string getInfo() const {
28         return carMake + " " + carModel + " " + licensePlate;
29     }
30 private: //SC6
31     string carMake;
32     string carModel;
33     string licensePlate;
34 };
35
36 class Payment {
37 public: //SC7
38     Payment(double ticketPayment, const string& creditCard) :
39         ticketPayment(ticketPayment), creditCardNumber(creditCard) {}
40     string getPaymentInfo() const {
41         stringstream ss; //8
42         ss << "$" << fixed << setprecision(2) << ticketPayment << " " <<
43             creditCardNumber;
44         return ss.str();
45     }
46 private: //SC9
47     double ticketPayment;
48     string creditCardNumber;
49 };
50
```

```
..._State\Grader Files\CPP_State\CPP_State\CPP_State.cpp 2
47
48 class Customer {
49 public: //SC10
50     Customer(const Name& n, const CarInfo& c, const Payment& p) : name(n), ↵
        carInfo(c), payment(p) {}
51     void printInfo() const { //SC11
52         cout << "Name: " << name.getFullName() << endl;
53         cout << "Car Information: " << carInfo.getInfo() << endl;
54         cout << "Speeding Ticket Payment: " << payment.getPaymentInfo() << ↵
            endl;
55         cout << endl;
56     }
57 private: //SC12
58     Name name;
59     CarInfo carInfo;
60     Payment payment;
61 };
62
63 // Driver class
64 class TicketDriver {
65 public:
66     void run() { //SC13
67         readDataFromFile();
68         printRecords();
69     }
70
71 private:
72     void readDataFromFile() { //SC14
73         ifstream file("Names.txt");
74         if (!file.is_open()) {
75             cerr << "Error opening file." << endl;
76             return;
77         }
78
79         string line;
80         while (getline(file, line)) {
81             stringstream ss(line);
82             string firstName, lastName, carMake, carModel, licensePlate, ↵
                creditCardNumber;
83             double ticketPayment;
84
85             if (getline(ss, firstName, ',') && getline(ss, lastName, ',') ↵
                && //SC15
86                 getline(ss, carMake, ',') && getline(ss, carModel, ',') &&
87                 getline(ss, licensePlate, ',') && ss >> ticketPayment && ↵
                    ss.ignore() &&
88                 getline(ss, creditCardNumber)) {
89                 Name name(firstName, lastName);
90                 CarInfo carInfo(carMake, carModel, licensePlate);
```

```
..._State\Grader Files\CPP_State\CPP_State\CPP_State.cpp 3
91         Payment payment(ticketPayment, creditCardNumber);
92         customers.emplace_back(name, carInfo, payment);
93     }
94     else {
95         cerr << "Error parsing line: " << line << endl; //SC16
96     }
97 }
98
99     file.close(); //SC17
100
101     void printRecords() const {
102         int lineNumber = 1;
103         cout << "*****" << endl;
104         cout << "**   BPA Speeding Ticket Database   **" << endl;
105         cout << "*****" << endl;
106         cout << "*****" << endl;
107         cout << "                               " << endl;
108
109         for (const auto& customer : customers) {
110             cout << "*****>>>  " << setw(2) << lineNumber << "      ↗
111                 <<<*****" << endl; //SC19
112             customer.printInfo(); //SC18
113             lineNumber++;
114         }
115         cout << "*****" << endl;
116     }
117     vector<Customer> customers;
118 };
119
120 int main() {
121     TicketDriver driver;
122     driver.run();
123     return 0;
124 }
125
```